

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

30/06/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with support for a **software-emulated division operator**.

The software-emulated division operator shall have the same functional behavior of regular division for positive dividend and divisors. However, *no DIV instructions* must be emitted by ACSE to translate it to assembly language. The operation may have undefined behavior if either or both dividend and divisor are negative — i.e. the implementation does not need to handle that case. Division by zero must be handled by computing a quotient of $2^{31} - 1$. The proposed implementation should perform proper constant folding. The code executed at compile time is not restricted in its use of division instructions. Therefore the C division operator may be freely used in the implementation, while the following functions are not allowed:

- `gen_div_instruction()`,
- `gen_divi_instruction()`,
- `handle_bin_numeric_op()` when the operator argument is `DIV`.

There are no constraints imposed regarding the run-time complexity of the software-emulated division.

The symbol associated to this new operator is a bracketed slash (`[/]`) instead of the familiar slash (`/`). Its precedence and associativity are the same as non-emulated standard division, which is retained and coexists with the new software-emulated one.

The following code snippet exemplifies the use of the new operator. As long as both a and b are positive and $b \neq 0$, the snippet will always print 1 when executed.

```
int a, b;

read(a);
read(b);
if (a / b == a [/] b) {
    write(1);
} else {
    write(0);
}
```

Tips: The simplest algorithm for computing division is called *repeated subtraction*. It involves subtracting the divisor from the dividend until the dividend becomes smaller than the divisor. The quotient is given by the number of subtractions performed.

In your solution you can write `INT_MAX` to refer to the constant $2^{31} - 1$.

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..5481e8f 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -69,6 +69,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"-"          { return MINUS; }
"*"          { return MUL_OP; }
"/"          { return DIV_OP; }
+ "[/]"      { return SW_DIV_OP; }
"&"          { return AND_OP; }
"|"          { return OR_OP; }
"!"          { return NOT_OP; }

diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..alb141c 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -16,6 +16,7 @@
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
+ #include <limits.h>
#include "axe_struct.h"
#include "axe_engine.h"
#include "symbol_table.h"
@@ -125,6 +126,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+ %token SW_DIV_OP

%token <label> DO
%token <while_stmt> WHILE
@@ -153,7 +155,7 @@ extern void yyerror(const char*);
%left LT GT LTEQ GTEQ
%left SHL_OP SHR_OP
%left MINUS PLUS
- %left MUL_OP DIV_OP
+ %left MUL_OP DIV_OP SW_DIV_OP
%right NOT_OP

/* =====
@@ -544,6 +546,67 @@ exp: NUMBER      { $$ = create_expression ($1, IMMEDIATE); }
                                (program, exp_r0, $2, SUB);
                                }
+
+ | exp SW_DIV_OP exp
+ {
+     if ($3.expression_type == IMMEDIATE && $3.value == 0) {
+         // If the divisor is constant and equal to zero, constant-fold the
+         // expression to (2^31)-1.
+         $$ = create_expression(INT_MAX, IMMEDIATE);
+     } else if ($1.expression_type == IMMEDIATE && $3.expression_type == IMMEDIATE) {
+         // If both operands are constants we know the result at compile time.
+         $$ = create_expression($1.value / $3.value, IMMEDIATE);
+     }
+ }
```

```

+
+     } else {
+         // Otherwise at least one operand is not a constant. Generate code
+         // which implements the repeated subtraction algorithm at run-time.
+         // Reserve the register that will hold the result (quotient).
+         int r_quot = getNewRegister(program);
+         $$ = create_expression(r_quot, REGISTER);
+
+         // Reserve a label for continuing to the following code
+         t_axe_label *l_exit = newLabel(program);
+
+         // If the divisor is a constant, we already checked at the beginning
+         // that it is not equal to zero. Otherwise, we need to generate the
+         // code to check at runtime.
+         if ($3.expression_type == REGISTER) {
+             gen_subi_instruction(program, REG_0, $3.value, 0);
+             t_axe_label *l_notDivZero = newLabel(program);
+             gen_bne_instruction(program, l_notDivZero, 0);
+             gen_move_immediate(program, r_quot, INT_MAX);
+             gen_bt_instruction(program, l_exit, 0);
+             assignLabel(program, l_notDivZero);
+         }
+
+         // Generate code to initialize the quotient to zero.
+         gen_move_immediate(program, r_quot, 0);
+         // Generate code to copy the dividend into a new register. This
+         // register will be diminished by the divisor at each iteration of the
+         // loop.
+         int r_rem = getNewRegister(program);
+         if ($1.expression_type == IMMEDIATE) {
+             gen_move_immediate(program, r_rem, $1.value);
+         } else {
+             gen_addi_instruction(program, r_rem, $1.value, 0);
+         }
+         // Generate the loop that subtracts the divisor from the dividend
+         // until the divisor is smaller than the dividend. The number of
+         // iterations is counted by r_quot.
+         t_axe_label *l_loop = assignNewLabel(program);
+         if ($3.expression_type == IMMEDIATE) {
+             gen_subi_instruction(program, r_rem, r_rem, $3.value);
+         } else {
+             gen_sub_instruction(program, r_rem, r_rem, $3.value, CG_DIRECT_ALL);
+         }
+         gen_blt_instruction(program, l_exit, 0);
+         gen_addi_instruction(program, r_quot, r_quot, 1);
+         gen_bt_instruction(program, l_loop, 0);
+
+         // Assign the exit label.
+         assignLabel(program, l_exit);
+     }
+ }
+
+ ;
+
+ %%
diff --git a/tests/sw_div/sw_div.src b/tests/sw_div/sw_div.src
new file mode 100644
index 0000000..3ccf783
--- /dev/null
+++ b/tests/sw_div/sw_div.src
@@ -0,0 +1,25 @@
+int a, b;
+a = 1234;
+b = 33;
+
+write(1234 / 33);
+write(1234 [/] 33);
+
+write(1234 / b);
+write(1234 [/] b);

```

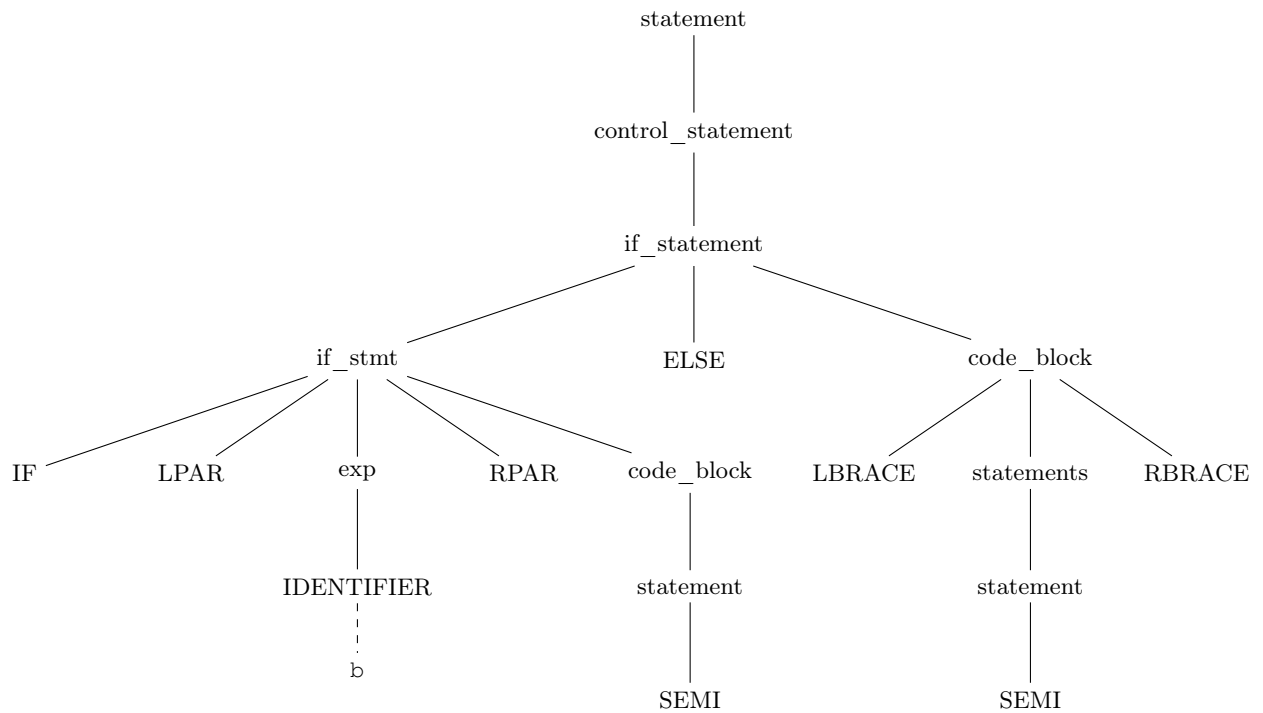
```
+  
+write(a / 33);  
+write(a [/] 33);  
+  
+write(a / b);  
+write(a [/] b);  
+  
+b = 0;  
+write(a [/] 0);  
+write(1234 [/] 0);  
+write(a [/] b);  
+  
+read(a);  
+read(b);  
+write(a / b);  
+write(a [/] b);
```

4. Given the following Lance code snippet:

```
if (b) ; else { ; }
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the statement nonterminal*. (5 points)

Solution



5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the software-emulated division in order to add support for negative dividends and negative divisors.